# 本科毕业论文

# 面向持久内存的文件系统设计与实现

基于 xv6 的 PMFS v3：受限 /pm、COW 提交与可复现验收

专业名称： 计算机科学与技术

学生姓名：（姓名）

指导教师：（指导教师）

所在学院：（学院）

完成日期： 二〇二六 年 六 月

（按学校正式封面模板替换本页）

# 摘　要

持久内存（Persistent Memory, PMEM）以字节寻址提供近似 DRAM 的访问延迟并在掉电后保留数据，对以块设备与日志为中心的操作系统存储栈提出了新的一致性模型与验收问题。教学操作系统 xv6 以 virtio 与日志型文件系统为根，适合讲授经典 UNIX 语义，但难以直接承载「声明假设下的可复现提交验收」。

本文按原题四要件，在 xv6-riscv 上实现 **PMFS v3**：保留根文件系统，经固定挂载点 `/pm` 提供受限文件子系统；以 COW 元数据与不可变 KV run 组织字节寻址布局，使文件数据路径不经 `bio`/`log`/`virtio`；以行级 flush 请求、可选 Zicbom `CBO.CLEAN` 与双槽 checkpoint 提交词实现轻量提交协议；为 QEMU 9.2.0 提供实验性独立 NVDIMM 补丁路径。语义上写死可见点与 POSIX 差集：仅成功 `fsync`/checkpoint 后承诺完整版本可读回。

核心贡献是教学 OS 上**可复现的提交验收方法学**（进程重启切点 oracle、有界规格负控、轨迹采样）及受限但可测的文件子系统实现。实验分两套口径：架构面报告对齐工作量下相对原生 `fs+log` 的延迟/短批吞吐；公平面报告同契约软件请求消融；并明确设备模型与持久语义不等价。

**关键词：**持久内存；NVDIMM；文件系统；崩溃一致性；写放大；xv6；契约测试；COW

# Abstract

Persistent memory (PMEM) offers byte-addressable access with near-DRAM latency while retaining data across power loss, challenging OS storage stacks built around block devices and journaling. The teaching OS xv6 provides a virtio-backed log file system as root; it is excellent for classic UNIX semantics but does not by itself provide reproducible acceptance of PM commit protocols under explicit assumptions.

This thesis implements **PMFS v3**, a bounded persistent-memory file subsystem on a fixed `/pm` mount beside the xv6 root: COW metadata and immutable KV runs without the block I/O stack; line-granular flush requests with optional Zicbom `CBO.CLEAN`; dual-slot checkpoints; and an experimental QEMU 9.2.0 NVDIMM patch path. Visibility is stated explicitly: a successful `fsync` publishes a complete version; unsynchronized updates are not promised after failure.

The distinctive contribution is a **reproducible commit-acceptance methodology** (process-kill cut-point oracles, bounded specification negatives, sampled traces) plus a measurable teaching-OS file subset. Evaluation separates an architectural plane (matched-workload latency/throughput versus xv6 `fs+log`, with unequal persist contracts) from a fairness plane (same-PMFS software-request ablations).

**Keywords:** persistent memory; NVDIMM; file system; crash consistency; write amplification; xv6; contract testing; copy-on-write

# 目录

**摘要** **I**

**Abstract** **II**

**第一章 绪论** **1**
1.1 研究背景 . . . . . 1
1.2 题面与问题陈述 . . . . . 1
1.3 研究内容与目标 . . . . . 1
1.4 主要贡献 . . . . . 2
1.5 论文结构 . . . . . 2

**第二章 相关工作** **3**
2.1 教学操作系统中的块文件系统 . . . . . 3
2.2 持久内存文件系统与编程模型 . . . . . 3
2.3 崩溃测试与一致性验收 . . . . . 3
2.4 日志结构、切分与索引 . . . . . 3
2.5 作者前序工作与工程契约 . . . . . 3
2.6 定位小结 . . . . . 3

**第三章 系统设计** **5**
3.1 产品形态与非目标 . . . . . 5
3.2 地址空间与后端 . . . . . 5
3.3 软件分层与对象图 . . . . . 5
3.4 崩溃可见点与 POSIX 差集 . . . . . 6
3.5 刷写与 checkpoint 发布顺序 . . . . . 6
3.6 维护、CDC 与评测意图 . . . . . 6

**第四章 系统实现** **7**
4.1 模块组织 . . . . . 7
4.2 VFS 与普通文件路径 . . . . . 7
4.3 COW 元数据、run 与 seal . . . . . 7
4.4 刷写、设备与可选 CBO . . . . . 7
4.5 维护：页复用与显式压缩 . . . . . 8
4.6 验收脚手架 . . . . . 8

**第五章 实验与分析** **9**
5.1 实验环境与口径 . . . . . 9
5.2 功能、恢复与模型（尖刺证据） . . . . . 9
5.3 架构面：对齐工作量的延迟与吞吐 . . . . . 9
5.4 公平面：软件请求与提示消融 . . . . . 10
5.5 维护与空间 . . . . . 10
5.6 实现难点与失败史 . . . . . 12
5.7 局限与威胁效度 . . . . . 12
5.8 本章小结 . . . . . 12

**第六章 结论与展望** **13**
6.1 工作总结 . . . . . 13
6.2 不足 . . . . . 13
6.3 展望 . . . . . 13

**附录 A 复现命令与设备说明** **16**
A.1 安全警告 . . . . . 16
A.2 推荐验收命令 . . . . . 16
A.3 关于「NVDIMM」与「掉电」 . . . . . 16
A.4 论文 PDF 编译 . . . . . 16

**致谢** **17**

# 第 1 章　绪论

## 1.1　研究背景

操作系统存储子系统长期以块设备为中心：文件内容经缓冲缓存、日志与块 I/O 栈落盘，崩溃一致性依赖有序刷写与检查点。教学操作系统 xv6 [3] 以 virtio-blk 与日志型文件系统讲授这一经典路径，语义清晰、代码可阅读，但延迟与吞吐受块设备与元数据日志路径约束，且难以直接表达字节寻址持久内存（Persistent Memory, PMEM）上的提交与故障模型。

PMEM 以字节寻址提供近似 DRAM 的访问延迟, 并在掉电后保留数据 [6]。学术界已出现面向 PMEM 的文件系统与编程模型,如 PMFS [4]、NOVA [9]、BPFS [2]、Mnemosyne [8] 等。它们多面向通用或接近生产的 Linux 环境。将完整 POSIX VFS 移植进 xv6 既超出本科毕设合理体量，也易掩盖「如何声明假设并验收提交正确性」这一更尖锐的工程问题。

## 1.2　题面与问题陈述

原题要求在 xv6 内核中设计原生支持持久化内存的文件系统，核心包括四项：（1）改造内存管理以映射 QEMU 模拟的 NVDIMM；（2）重构元数据与数据布局以适配字节寻址并消除传统块设备抽象层；（3）设计基于缓存行刷写的轻量崩溃一致性机制以保障掉电安全；（4）在 QEMU 中完成集成，并通过对比测试验证相对原生文件系统在延迟与吞吐上的架构级优势。

本文将上述要求落实为三个相互耦合的研究问题：

1. **形态：**在保留 virtio 根的前提下，如何提供可经普通文件 API 访问的 PM 文件路径（固定挂载点 `/pm`），而非声称完整 POSIX/VFS?
2. **一致性：**COW 元数据、不可变 run 与双槽 checkpoint 如何共同保证「成功 `fsync` 后可读回完整版本」? 平台刷写与掉电假设如何显式陈述?
3. **验收与评测：**如何用可复现的进程重启切点、有界规格模型与对齐工作量基准，分别支撑正确性与架构级性能叙事?

## 1.3　研究内容与目标

本文设计并实现 **PMFS v3**：并行于根文件系统的受限 PM 文件子系统。主要内容包括：

- 后端：EMUL、普通 MAP（guest RAM）、共享文件后端，以及面向 QEMU 9.2.0 的**实验性独立 NVDIMM** 补丁与私有 FDT 发现；可选 Zicbom `CBO.CLEAN`；

- 布局：不可变 KV run + COW inode/dentry；双槽 checkpoint 以对齐 8 B 提交词发布根；数据路径不经 `bio`/`log`/`virtio`；
- 语义：`/pm` 上普通 `open`/`read`/`write`/`lseek`/`fsync`、目录、硬链接与子树 `rename`；写死 POSIX 差集与资源上限；
- 验收：进程重启故障切点 oracle、有界协议模型负控、源级轨迹重放；
- 评测：同 API、同成功数、内容读回校验的延迟/短批吞吐；同 PMFS 内软件请求消融。

## 1.4 主要贡献

贡献表述为可核验的工程组合与方法学，**不**声称算法首创 [11, 10, 12]：

**C1 方法学**
教学 OS 上可复现的提交验收：QEMU 进程重启切点、损坏拒绝挂载、有界规格负控与轨迹采样；相对「演示双槽 UI」，强调可脚本判定。

**C2 系统**
受限 VFS+`/pm`、COW/KV/CDC 协同、双槽保护下的页复用与显式压缩；行级 flush 请求与可选 CBO。

**C3 实验**
对齐工作量的架构面延迟/吞吐，以及同契约软件消融；明确持久语义不等价与设备模型边界。

主动不宣称：完整内核 VFS 替换根、上游通用 NVDIMM/NFIT、无假设的物理掉电证明、真实器件磨损改善、完整 POSIX。

## 1.5 论文结构

第 二 章综述相关工作；第 三 章给出设计；第 四 章描述实现；第 五 章报告实验与威胁效度；第 六 章总结。附录给出复现命令与设备路线。

# 第 2 章　相关工作

## 2.1　教学操作系统中的块文件系统

xv6 [3] 的 `fs+log` 以 inode、目录项、位图与日志检查点保证崩溃后可恢复。其 I/O 绑定 virtio 块设备与 bio；原生 `fsync` 主要等待全局日志提交，驱动未协商等价的设备 flush。本文将该路径作为**架构面**对照，并**保留为根文件系统**：PMFS 经固定挂载点 `/pm` 并行接入。

## 2.2　持久内存文件系统与编程模型

PMFS [4] 与 NOVA [9] 面向 PMEM 提供接近生产的文件语义；Strata [5] 探索跨介质分层；BPFS [2]、Mnemosyne [8] 强调细粒度持久与编程抽象。本文取其「字节寻址 + 显式持久」动机，但收敛为教学体量的**可测子集**：固定挂载、有界资源、COW 对象图与 checkpoint，而非 Linux VFS 替身。

## 2.3　崩溃测试与一致性验收

CrashMonkey、PMTest、Witcher 等研究自动探索崩溃与一致性缺陷。本文不声称首创该类方法，而是在 xv6+QEMU 上构建**进程重启切点 oracle**、有界手写规格模型与源级写范围轨迹重放，使提交协议可复现、可负控。验收的是「声明故障模型下的协议行为」，不是物理掉电或全路径形式化证明。

## 2.4　日志结构、切分与索引

LSM 类结构 [7] 与 Bloom 过滤 [1] 广泛用于写入友好路径。本文 run 不可变、批量 seal、Bloom/fence 与有界 posting 属于同一设计家族；CDC 用于有界切分（无内容去重）。实现服务教学内核约束：页粒度、有限段数、粗粒度锁。

## 2.5　作者前序工作与工程契约

作者前序 **FCS** [11] 提供双槽、不可变 run、写放大等约束；**Aurora** [10] 提供量化键与 posting 等索引约束。二者源码不并入本仓库，xv6 侧为 C 自研重写。`ONBOX_*` 串口契约手法 [12] 在本课题扩展为阶段码、recovery 脚本与 CI 目标。

## 2.6　定位小结

表 2.1　相关工作与本文关系

| 工作 | 问题域 | 与本文 |
| --- | --- | --- |
| xv6 fs+log | 教学块 FS | 架构面对照；仍作根 |
| Linux PMFS/NOVA/BPFS | 生产/研究 PMEM FS | 动机参考；本文为可测子集 |
| 崩溃测试系统 | 自动一致性探索 | 方法学对照; 本文为 QEMU 切点 oracle |
| FCS / Aurora | 耐久引擎与索引（前序） | 约束下沉；代码自研 |
| **本文 PMFS v3** | 教学 OS 上 /pm+COW+ 验收 | **尖刺 = 可复现提交验收** |

# 第 3 章　系统设计

## 3.1　产品形态与非目标

PMFS v3 的产品形态是：**保留 virtio 根，经固定合成挂载点 /pm 提供受限文件子系统**。用户可用普通 `open/read/write/lseek/fsync/getdents` 等访问 PM 文件；并保留专用 KV/stream 接口。**不**替换根 FS，**不**做动态 mount table 或完整 POSIX。

非目标：上游通用 NVDIMM/NFIT/热插拔；无平台假设的物理掉电证明；符号链接、权限、`mmap`、跨后端 rename、打开后删除最后链接；全设备 GC/硬件磨损均衡。

## 3.2　地址空间与后端

Listing 3.1　地址布局示意（MAP / 实验 NVDIMM）

```
KERNBASE ---------- 0x80000000
  kernel + user（易失 DRAM，典型 128MiB）
PHYSTOP  ---------- KERNBASE + 128MiB
PMEMBASE ---------- PHYSTOP（如 0x88000000）
  PMFS 窗口：双槽 checkpoint / 元数据 / run
PMEMEND  ---------- PMEMBASE + 32MiB（名义；MAP 受 DTB 截断）
```

表 3.1　物理后端与可验证范围

| 模式 | 含义 | 不可据此声称 |
|---|---|---|
| EMUL | 8 MiB 静态易失缓冲 | 跨启动保留、硬件持久 |
| 普通 MAP | `PHYSTOP` 后 guest RAM | 独立 NVDIMM 设备 |
| 共享 RAM 文件 | file-backend，便于杀进程重启 | 宿主掉电、独立设备 |
| 实验 NVDIMM | QEMU 9.2.0 补丁冷插拔 `nvdimm`+ 私有 FDT | 上游通用 virt、NFIT、物理掉电 |
| Zicbom（可选） | 发出 `CBO.CLEAN` | TCG 下的真实缓存写回 |

MAP 在窗口不足或设备缺失时**拒绝挂载**，不静默降级为 EMUL。

## 3.3　软件分层与对象图

自上而下：VFS 逐组件遍历（原生 inode 与 PM inode pin）→ `pmfile`（共享工作缓冲、COW 元数据）→ `pmstream/pmcdc` → `pmalgo/pmindex` → `pmfs`（双槽 checkpoint、可达性、页复用）→ `pmdev/pmem`。

文件内容以 KV chunk（extent）承载；元数据为不可变快照页。容量边界（量级）：约 40 inode/dentry、每文件 4 extent、文件 $\leq$1024 B。

## 3.4 崩溃可见点与 POSIX 差集

表 3.2 提交可见性（软件协议）

| 操作 | 故障后承诺 | 说明 |
|---|---|---|
| 未成功 `fsync` 的写 | **不承诺**保留 | 工作缓冲/memtable |
| 成功 `fsync` | 完整新版本可读回 | 数据 + 元数据 + 提交词 |
| 空白/v2/校验损坏 | 拒绝挂载 | 不自动 format |
| 单槽损坏 | 可回退另一槽 | 双槽均坏则失败 |

差集包括：动态挂载、完整跨目录/跨后端 rename、符号链接、权限、`mmap`、多写者线性化证明等。硬链接、`lseek`/`O_APPEND`、PM 内子树 rename、`getdents` 已支持（见实现章）。

## 3.5 刷写与 checkpoint 发布顺序

持久化请求语义依赖平台：默认 `pmem_flush` 为按行 touch+`fence`（可观测、可排序）；可选 Zicbom 发出 `CBO.CLEAN`。一次根发布的设计顺序为：

1. 追加 run / 分配 COW 元数据页；对各页请求 persist + fence；
2. 非当前槽：清提交词并 persist；写入新头（未提交）并 persist；
3. **一次 8 B store** 写入提交状态与校验 $\rightarrow$ persist + fence；再更新内存 epoch/槽。

该协议还依赖：屏障顺序、提交词故障原子性、映射有效、校验未碰撞等假设。QEMU TCG 不模拟缓存持久域，故正文将「掉电安全」表述为**声明假设下的协议正确性**，并以进程重启 oracle 验证。

## 3.6 维护、CDC 与评测意图

双槽可达性保护两代根；双槽均不可达页可复用。显式 `pmcompact` 做全链 COW 压缩，近满或缺暂存空间可失败。普通写优先 CDC，超预算则固定分块；**无内容去重**。

评测分两平面：**架构面**——相对 xv6 的同工作量延迟/吞吐（持久契约不等价）；**公平面**——同 PMFS 内软件请求/策略消融。禁止混读。

# 第 4 章　系统实现

## 4.1　模块组织

实现基于 xv6-riscv。主要模块包括：VFS 路径遍历与 /pm 挂载点；pmfile（普通 fd、COW 元数据、硬链接/rename）；pmcdc/pmstream；pmalgo/pmindex；pmfs（checkpoint、可达性、压缩）；pmdev/pmem；以及 tools/pmfs_recovery.py、test-xv6.py、契约与 bench 脚本。文件内容经 pm_put/seal 落入 run；元数据经 COW 页与 checkpoint 发布。

## 4.2　VFS 与普通文件路径

vfs_walk 逐组件解析；仅原生根下的 pm 进入 PMFS，/pm/.. 返回原生根。PM inode pin 支持 cwd/目录 fd；独立 open 各自维护 offset，共享工作缓冲；dup/fork 共享打开对象。PMFS sleep lock 串行化缓冲、append 取尾与同步。

硬链接维护共享 inode 与链接数；非最后链接可在打开时删除，最后链接的 unlink-open 仍拒绝。子树 rename 检查环与路径长度后，修改内存图并以一次 checkpoint 发布；非替换可复用不可变 inode 页。

## 4.3　COW 元数据、run 与 seal

内容同步：CDC/固定切分 → pm_put → seal 追加不可变 run → 新 inode 快照（可复用未变 dentry）→ checkpoint。pm_get 优先 memtable/缓存，再精确索引或 newest-first + Bloom/fence。索引满后未跟踪键走 fallback；更新已跟踪键必须刷新指向。

安全挂载：无有效/旧格式/损坏则失败关闭；仅 pmformat --force 显式格式化。v2 与 v3 不兼容，无迁移工具。

## 4.4　刷写、设备与可选 CBO

pmdev_write_page 后调用 pmem_persist。默认 flush 逐 64 B 行 touch 并计数；pmem_fence 含编译器屏障与内存 fence。可选 PMEM_ZICBOM 发出 CBO.CLEAN（假定块大小与权限）。实验性 NVDIMM：补丁 QEMU 注册独立设备，内核在 kalloc 前解析私有 FDT 并校验地址/大小。

## 4.5 维护：页复用与显式压缩

可达性扫描覆盖两槽根与 run 链；不可达页可分配给新元数据或新 run。`pmcompact` 要求 memtable/stream 空，预留不相交暂存后写出新链并一次发布；忙或空间不足返回明确错误。这不是在线全设备 GC。

## 4.6 验收脚手架

**旧自测**

`ONBOX_PMFS_*` 阶段码；`contract-full` 等破坏性，仅 disposable。

**重启 oracle**

`make test-recovery*`：临时后端、guest 步骤、`SIGKILL`、同后端读回；覆盖挂载拒绝、文件/KV、目录/链接、插桩切点。

**模型**

`make test-model`：有界协议枚举 + 负控反例。

**轨迹**

测试构建记录写范围/flush/fence，采样重放提前逐出等。

尖刺价值：正确性依赖「可脚本判定 + 诚实假设」，而非口头演示。

# 第 5 章　实验与分析

## 5.1　实验环境与口径

表 5.1　实验环境（代表性配置）

| 项 | 配置 |
|---|---|
| 基线 | xv6-riscv + PMFS v3 |
| 模拟器 | QEMU 9.2.0 `virt`（TCG） |
| 后端 | EMUL / MAP / 共享 RAM / 实验性 NVDIMM（补丁） |
| 刷写 | 默认 touch+fence；可选 Zicbom |
| 计时 | guest `r_time`；设备树 timebase 10 MHz（perf 记录） |

**两套结论：**架构面——相对 xv6 的同工作量软件路径量级差（持久契约**不等价**）；公平面——同 PMFS 内软件请求/策略效应。原始日志见仓库 `docs/results/`。

## 5.2　功能、恢复与模型（尖刺证据）

表 5.2　主要验收门

| 门类 | 命令/产物 | 可证明 / 不可证明 |
|---|---|---|
| 安全挂载 | recovery / repair 记录 | 拒绝空白/v2/损坏；非自动修复任意损坏 |
| 进程重启 oracle | `test-recovery*` | 同后端杀进程后读回；非物理掉电 |
| VFS/硬链接 | `test-vfs*` | `/pm` 语义子集；非完整 POSIX |
| 规格模型 | `test-model` | 有界协议 + 负控；非内核全轨迹证明 |
| 轨迹重放 | `test-trace-*` | 源级范围采样；非指令级 |
| 旧契约 smoke | `contract-full` 等 | 阶段码；破坏性 |

分析：相对仅展示双槽结构，可脚本化的切点与负控构成方法学差异。实验性 NVDIMM 路径另以 QMP 核验设备类型/地址/独立后端，并跑通同类 oracle；仍不能外推宿主掉电。

## 5.3　架构面：对齐工作量的延迟与吞吐

对比两侧使用普通文件 API、相同成功操作数、内容与 EOF 校验；路径深度对齐。PMFS 样本取自 EMUL（易失），默认 flush 为 touch+fence；xv6 `fsync` 为全局日志同步

且无设备 flush。故结果只支持**去块日志栈的架构级软件路径**叙事。

表 5.3　同工作量 p50 延迟与短批吞吐（摘录，QEMU TCG / 单 hart）

| 负载 | 字节 | PM p50(ms) | xv6 p50(ms) | PM txn/s | xv6 txn/s |
|---|---|---|---|---|---|
| roundtrip | 8 | 2.140 | 21.812 | 467.3 | 45.2 |
| roundtrip | 1024 | 2.362 | 21.989 | 444.0 | 43.0 |
| overwrite | 8 | 1.476 | 7.479 | 704.7 | 125.7 |
| overwrite | 1024 | 1.791 | 7.437 | 583.6 | 130.5 |

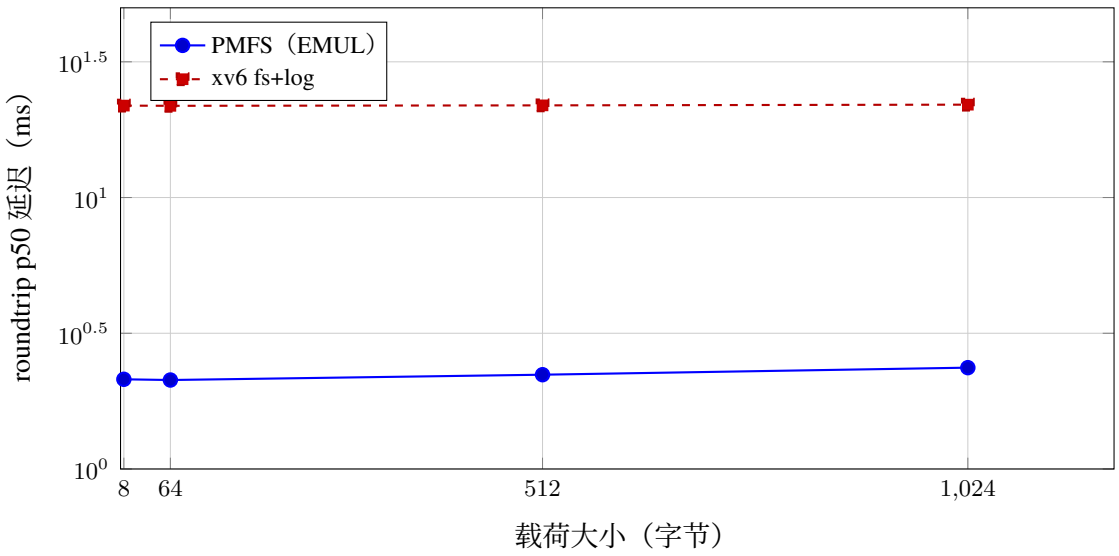



图 5.1　对齐工作量 roundtrip 的 p50 延迟（QEMU TCG，单 hart）。两侧 API/成功数/读回校验对齐，但持久契约不等价，仅支持架构级软件路径叙事。

观察：roundtrip 上 PM 相对 xv6 约一个数量级延迟优势；短批吞吐同数量级差。PM 大块覆写吞吐低于小块，反映软件写路径成本。**禁止**写「真实持久内存千倍加速」。

## 5.4　公平面：软件请求与提示消融

同 PMFS、同一 `fsync` 契约下，关闭可达性提示显著增加软件读请求并降低短批吞吐（64 B 覆写吞吐比约 4.2×，样本有限）。页写请求字节在两组相同，故消融**不**证明降低物理写入。

效率表（非替换 rename 页写 2→1、内容同步 3→2 等）见仓库 efficiency 结果；均为指定负载内核计数。

## 5.5　维护与空间

双槽保护下的页复用使指定元数据负载尾部高水位可控；显式压缩可归并 run，但需暂存空间，满盘可安全失败。软件写字节计数不是 NAND 擦除或器件寿命。

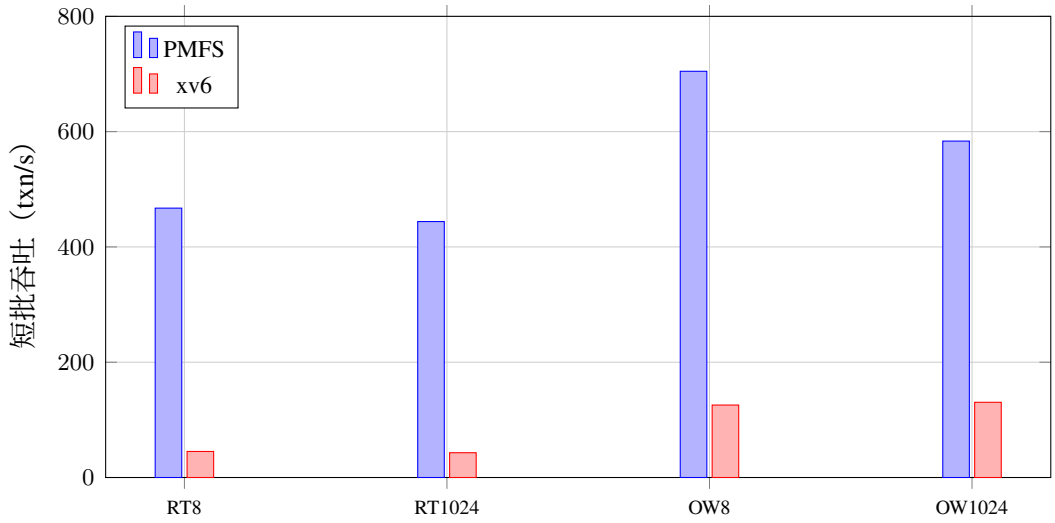



图 5.2　短批串行吞吐对比（RT=roundtrip，OW=overwrite）。非长时稳态、非墙钟 MB/s。

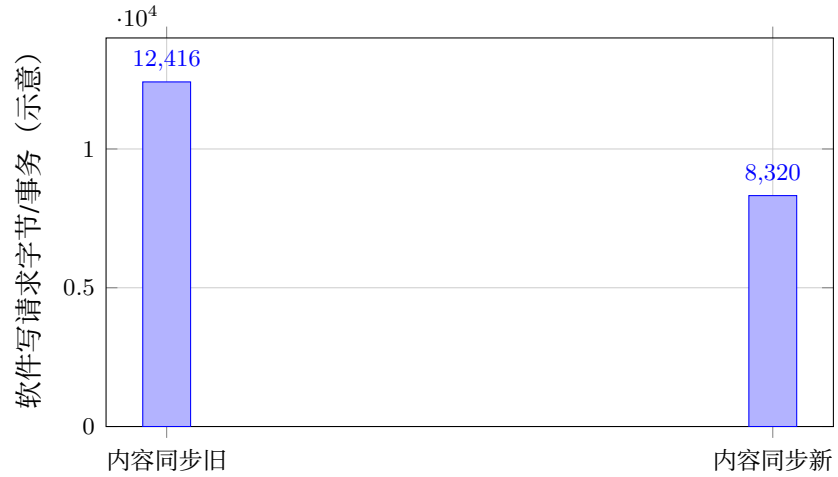



图 5.3　同负载下内容同步的软件写入字节计数对比（efficiency 记录；非介质磨损）。




图 5.4　同 PMFS 契约下可达性提示消融的吞吐比（配对 bootstrap 点估计；样本有限）。

## 5.6 实现难点与失败史

早期基线存在：破坏性 boot 自测改写介质、无条件自动 format、flush 仅遍历、跨进程文件读回失败、旧 bench 工作量不对齐等。v3 改为显式 format、安全挂载、COW 根、重启 oracle 与对齐 bench。细节见仓库踩坑与沙箱记录。这些叙述用于证明系统经故障闭环调出，而非一次性写成。

## 5.7 局限与威胁效度

1. 设备：补丁 NVDIMM 非上游通用；MAP/共享 RAM 不是 NVDIMM。
2. 持久域：默认非硬件写回；TCG 无缓存层级；杀进程 $\neq$ 物理掉电。
3. 性能：契约不等价；单环境 TCG 样本；外推真机需重校准。
4. 功能：有界资源与粗粒度锁；非完整 POSIX；近满压缩可失败。

## 5.8 本章小结

实验主线是可复现提交验收与诚实假设；性能服务于架构级软件路径说明；公平面消融不得与架构面 fratio 混读。

# 第 6 章　结论与展望

## 6.1　工作总结

本文在 xv6-riscv 上设计并实现了并行于根文件系统的 **PMFS v3**：固定挂载点 `/pm` 上的受限文件语义、COW 元数据与不可变 KV run、双槽 checkpoint 提交、行级 flush 请求与可选 Zicbom，以及面向 QEMU 的实验性独立 NVDIMM 补丁路径。系统以进程重启切点 oracle、有界规格负控与轨迹采样验收提交行为，并以对齐工作量基准报告相对原生 `fs+log` 的软件路径延迟/吞吐量级差。

主要结论：

1. **方法学：**不可替代点是可复现的验收资产与诚实假设清单，而非又一个双槽结构或完整生产 FS。
2. **系统：**成功 `fsync` 后可读回完整版本；空白/损坏拒绝挂载；PM 路径不经块日志栈。
3. **实验：**架构面展示量级优势但持久契约不等价；公平面展示软件请求效应，二者不可混读。

对照原题四要件：NVDIMM 映射在**实验补丁路径**有条件达标；去块布局实质达标；刷写一致性在**声明假设**下达标而物理掉电未证；架构级性能叙事在软件路径上成立。

## 6.2　不足

真机持久域校准、上游通用设备发现、完整 POSIX、可扩展并发、近满可持续 GC、以及论文正式模板化定稿仍不足。旧 PDF/历史不等量基准不得作为新结论。

## 6.3　展望

- 与导师书面确认设备模型与并行 `/pm` 产品形态；
- 将实际 store/flush 轨迹更紧地接入状态空间检查；
- 在保持尖刺的前提下扩展可测语义边界与论文图表重绘。

答辩一句话：**在教学 OS 上交付可经普通文件 API 访问的 PM 文件子系统，并以可脚本化故障验收与诚实假设证明提交协议；相对原生块日志展示架构级软件路径优势，但不声称无条件物理掉电或真机持久加速**。

# 参考文献


[1] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[2] Jeremy Condit, Edmund B. Nightingale, Christopher Frost, Engin Ipek, Benjamin C. Lee, Doug Burger, and Derrick Coetzee. Better I/O through byte-addressable, persistent memory. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, pages 133–146. ACM, 2009. 提出 BPFS.

[3] Russ Cox, M. Frans Kaashoek, and Robert Morris. *xv6: a simple, Unix-like teaching operating system*. MIT CSAIL, 2022. RISC-V edition; https://pdos.csail.mit.edu/6.1810/.

[4] Subramanya R. Dulloor, Sanjay Kumar, Anil Keshavamurthy, Philip Lantz, Dheeraj Reddy, Rajesh Sankaran, and Jeff Jackson. System software for persistent memory. In *Proceedings of the Ninth European Conference on Computer Systems (EuroSys)*, pages 1–15. ACM, 2014. 提出并实现 PMFS；EuroSys 论文题名为 System Software for Persistent Memory.

[5] Youngjin Kwon, Henrique Fingler, Tyler Hunt, Simon Peter, Emmett Witchel, and Thomas Anderson. Strata: A cross media file system. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 460–477. ACM, 2017.

[6] Hai-Kun Liu, Di Chen, Hai Jin, Xiao-Fei Liao, Binsheng He, Kan Hu, and Yu Zhang. A survey of non-volatile main memory technologies: State-of-the-arts, practices, and future directions. *Journal of Computer Science and Technology*, 36(1):4–32, 2021.

[7] Russell Sears and Raghu Ramakrishnan. bLSM: A general purpose log structured merge tree. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 217–228. ACM, 2012.

[8] Haris Volos, Andres Jaan Tack, and Michael M. Swift. Mnemosyne: Lightweight persistent memory. In *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 91–104. ACM, 2011.

[9] Jian Xu and Steven Swanson. NOVA: A log-structured file system for hybrid Volatile/Non-volatile main memories. In *14th USENIX Conference on File and Storage Technologies (FAST)*, pages 323–338, Santa Clara, CA, 2016. USENIX Association.

[10] 作者. Aurora：量化索引与查询缓存（作者前序原生项目）. 未公开发表设计文档与实现, 2024. 设计约束：键量化、二值 posting、查询 LRU；代码不并入本仓库.

[11] 作者. FCS：嵌入式耐久键值存储引擎（作者前序原生项目）. 未公开发表设计文档与实现, 2024. 设计约束：双槽超级块、不可变 run、B1/B2/B3、写放大计量；代码不并入本仓库.

[12] 作者. CXC/OS 工程化契约验收手法. 作者相关工程实践, 2025. ONBOX 串口契约思想；本课题重写为 ONBOX_PMFS_*.

# 第 A 章　复现命令与设备说明

## A.1　安全警告

`contract*`、`test-recovery*`、`bench-sweep`、部分 `test-xv6.py` 目标会 `make clean` 并重建 `fs.img`；boot 自测与 `pmformat --force` 会破坏介质。请仅在可丢弃 checkout 运行并先备份。

## A.2　推荐验收命令

```
make test-model
CPUS=3 make test-vfs
make test-recovery CPUS=3 RECOVERY_ARGS="--log-dir .ci-logs/recovery"
make test-recovery-zicbom CPUS=3 RECOVERY_ARGS="--log-dir .ci-logs/zicbom"
# 补丁 QEMU 后:
# python3 tools/check_nvdimm_device.py --qemu /path/to/qemu-system-riscv64
# make test-recovery-nvdimm QEMU=/path/to/qemu-system-riscv64
make contract-full           # 破坏性旧自测
python3 tools/bench_perf.py --runs 10 --warmup 1 --samples 32 \
  --output /tmp/pmfs-matched
```

文档索引见仓库 `docs/README.md`（含语义、契约、QEMU/NVDIMM 入口）。

## A.3　关于「NVDIMM」与「掉电」

- 未打补丁的 MAP / 整机共享 RAM **不是** NVDIMM 设备。
- 实验补丁提供独立 `nvdimm` 冷插拔与私有 FDT，可经 QMP 核验；非上游通用、无 NFIT/热插拔。
- 默认 `pmem_flush` 非硬件写回；可选 CBO 在 TCG 下仍无真实持久域。
- 进程 `SIGKILL` 验证的是该故障模型下的读回，**不是**物理掉电。

## A.4　论文 PDF 编译

```
cd thesis/latex
make          # xelatex + bibtex + xelatex x2
```

封面姓名/学院等宏在 `main.tex`；请替换为学校正式模板后再印刷提交。

# 致　谢

感谢指导教师在选题、系统边界与论文表述上的悉心指导。感谢教学操作系统 xv6 与 RISC-V / QEMU 开源社区提供的可读代码与实验平台。感谢课程与实验室同学在讨论、复现与答辩准备中的帮助。

本课题将工程验收（可机器判定契约）置于与数据结构同等重要的位置；文中踩坑记录旨在便于后来者复核。文责自负，疏漏之处恳请批评指正。